

AI Powered Audit Intelligence System

Technical & Technology Architecture, and Product Roadmap

PREPARED FOR

YES BANK – Internal Audit

PREPARED BY

Protiviti India

RFI REFERENCE

§3.2 Documentation · §6.1

DOCUMENT VERSION

1.0

STATUS

For technical evaluation

CLASSIFICATION

Confidential

1 How to read this document

This document answers three of the Bank's asks in one place: the **technical architecture document** and **high-level architecture diagram** required under §3.2 (Documentation and Expected Deliverables), and **"Architecture of Technology & Product RoadMap"**, which is point 1 of the nine-point techno-commercial submission under §6.

Two layers, always labelled

A reference implementation of this platform already exists and runs. It is not a slide-ware mock-up: it ingests data through real connectors, compiles and executes 78 controls against frozen evidence, and reproduces a run byte-for-byte on demand. It is also, today, a single-process reference environment.

The Bank's non-functional targets – 5,000+ concurrent users, 99.9% availability, DR RPO of zero and RTO of one hour – describe a production deployment, not a reference environment. Presenting one as the other would be misleading, and would not survive technical evaluation. Every architectural statement in this document is therefore tagged:

Tag	Meaning
AS-BUILT	Implemented and demonstrable today in the reference environment. Can be shown working in the prototype demonstration the RFI requests.
AS-PROPOSED	Part of the production architecture for YES BANK. Designed, sized and costed, but delivered during implementation – not demonstrable today.
PARTIAL	Substantively present, with a defined remainder. The remainder is stated explicitly rather than absorbed into a “yes”.

This mapping is exactly what §3.2 asks for under Expected Deliverables: “*implementation timelines for both ready-made and to-be-developed functionalities*” and “*clarification of items not feasible*”. §14 (Product Roadmap) sequences the as-proposed items; §15 states what is out of scope and what we need from the Bank.

COMPANION DOCUMENTS

This architecture is one of four technical deliverables. It is read alongside the **Technical & NFR Response Matrix** (clause-by-clause response to §3.2, §5 and the 30-point §6 checklist), the **Integration & API Documentation**, and the **Functional Requirements Matrix** covering all 228 functional requirements across the nine in-scope solutions.

2 Architecture at a glance

The platform is a single audit intelligence system with nine functional surfaces over one shared core. The nine initiatives in §3.1 are not nine products: they are nine views onto the same canonical data, the same control engine, the same evidence store and the same audit ledger. That is what makes cross-cutting requirements – a single exception register, one workflow, one set of dashboards, one security model – achievable rather than aspirational.

9 Functional surfaces on one shared core	11 Canonical entities, source-agnostic	12 Declarative control types – no code to author a control	117 REST API operations, OpenAPI documented
4 Connector types: file, SQL, SFTP, REST	100%		

The reference implementation comprises approximately 25,800 lines of Python across 69 modules and 6,450 lines of front-end code, validated by 649 automated unit tests and 126 end-to-end checks executed over HTTP against a running instance. Control accuracy is measured against a ground-truth dataset with planted violations: 66 of 66 controls match their expected result set exactly, all 963 planted violations are detected, and no near-miss record is wrongly flagged.

3 Architecture principles

Seven principles govern the design. Each is stated with the mechanism that enforces it, because a principle without a mechanism is a preference.

1 · Evidence is immutable and content-addressed

A control never runs against a live source system. It runs against a frozen snapshot whose identity is the hash of its contents.

Mechanism: each pull writes Parquet files and a manifest of per-table SHA-256 digests; the snapshot id is the SHA-256 of that manifest plus the source key. Re-verification re-hashes every file. Altering one byte of evidence changes the id and is detected.

2 · Controls are configuration, not code

A control is authored as a row in a workbook and validated against a published schema. Adding or amending a control requires no release.

Mechanism: twelve declarative rule types compile to a SQL predicate. The compiler never raises on an author's mistake — it returns the control as `unexecutable` with the reason, because a control that silently matches nothing is the worst failure mode in an audit system.

3 · AI reasons; code decides

Language models draft, explain and summarise. Every number that reaches a workpaper is computed by deterministic code.

Mechanism: model output is a proposal requiring human acceptance; the analytics path never calls a model. A generated control is compiled, validated and previewed against real data before it can be saved.

4 · The engine proposes; the auditor concludes

The system reports exceptions, populations and rates. It does not conclude that a control is ineffective – that judgement is the auditor's and is recorded as theirs.

Mechanism: a run yields a *proposed* conclusion and never returns “Ineffective”; where controls could not be executed the run is marked inconclusive rather than clean.

5 · Separation of duties is enforced, not documented

Maker and checker are different people at every gate that matters.

Mechanism: the platform rejects self-approval of a control, self-review of a finding, verification of an action by its owner or by whoever closed it, self-approval of a deadline extension, and promotion of a model by the person who trained it.

6 · Everything is on the record and tamper-evident

Every state change is written to an append-only ledger; nothing updates or deletes a ledger row.

Mechanism: each entry carries the hash of its predecessor, so any alteration breaks the chain from that point forward and is located by re-computation. The chain head is independently verifiable.

7 · Data stays where the Bank puts it

No audit data leaves the Bank's environment, including to an AI provider.

Mechanism: the inference layer accepts only loopback and private-host endpoints and refuses any non-local URL; an explicit configuration guard blocks external providers. Data residency is a property of the code, not of a policy statement.

4 Solution context

The platform sits between the Bank's systems of record and its audit function. It reads; it does not write back. No connector in the architecture has a write path to a source system, which removes an entire class of operational risk from the deployment conversation and simplifies the Bank's change-approval process for connecting it.

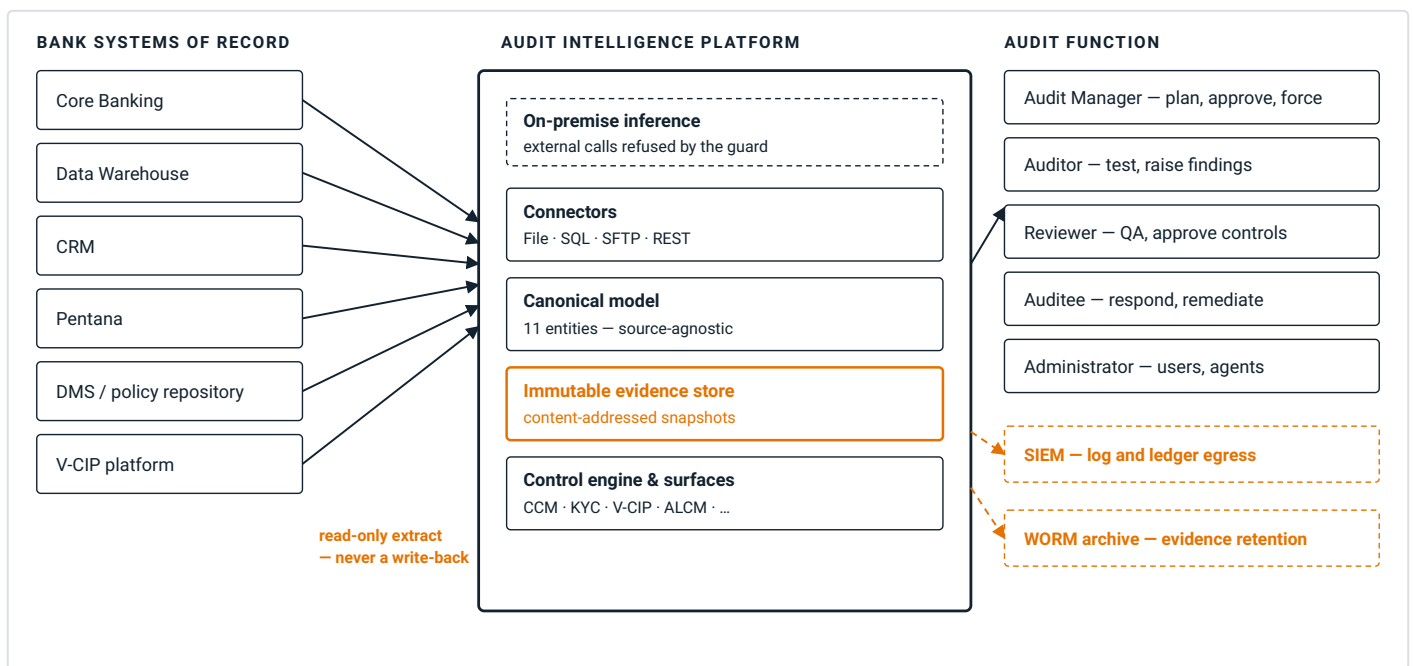


Figure 1 — Solution context. Source systems are read through one of four connector types and normalised into 11 canonical entities before anything is stored. The orange elements are the two that carry the assurance argument: evidence is content-addressed on the way in, and the audit ledger leaves the platform to the Bank’s SIEM and WORM archive on the way out. Dashed boundaries are **AS-PROPOSED**.

5 Logical architecture

The system is layered, and dependencies point one way only. No domain module imports the API layer; no data module imports a surface. This is what allows the control engine to be tested without a web server, and the ingestion pipeline to be exercised without a database — and it is why 649 automated tests can run in under a minute.

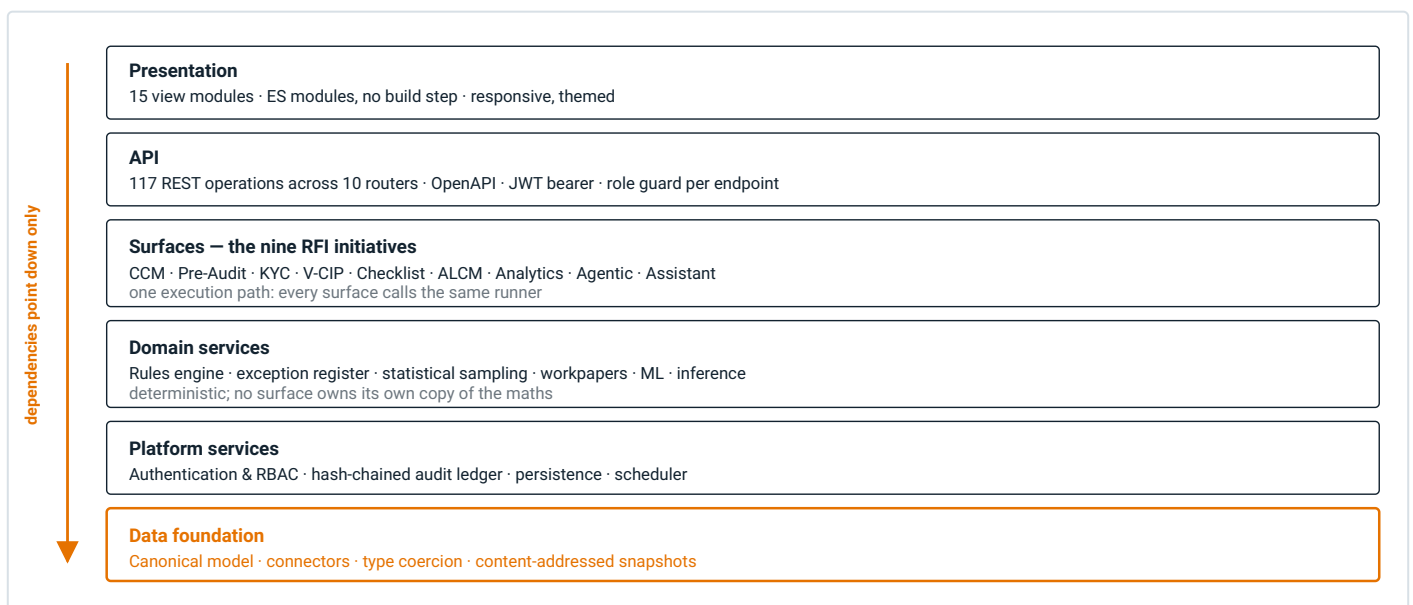


Figure 2 — Layered architecture. The nine initiatives named in §3.1 are the third layer, not nine systems. They share one execution path, one exception register and one ledger; the data foundation beneath them is what makes a control portable across source systems.

Module inventory

Package	Modules	Lines	Responsibility
aip/ingestion	5	1,905	Connectors, type coercion and rejection, content-addressed snapshots, schema-drift detection
aip/rules	6	3,267	Rule schema and validation, SQL compiler, execution engine, repository and versioning, workbook parser
aip/surfaces	5	2,218	Run orchestration, audit lifecycle, checklists, V-CIP scoring
aip/ml	5	1,679	Feature engineering (17 features), anomaly detection, exception triage, model registry
aip/platform	7	1,450	Authentication, RBAC, audit ledger, ORM (36 tables), scheduler, bootstrap
aip/sampling	2	817	Attribute, stratified, risk-weighted and monetary-unit sampling with reproducible selection
aip/ai	3	762	Inference provider with locality guard, natural-language control drafting and preview
aip/evidence	2	665	Workpaper rendering, hashing and sign-off
aip/exceptions	2	649	Exception store, deduplication, disposition, assignment, revalidation
aip/reporting	2	331	Export to XLSX, CSV and PDF
aip/workflow	2	287	Assignments, auditee queries, document attachment
api/routers	10	2,704	HTTP surface; 117 operations, every one carrying an explicit role dependency

6 Data architecture and evidence

The canonical model, and why it exists

A control in this platform never names a core-banking table. It names a canonical entity. `Dormant_Reactivation_No_Approval` is written against `account` , not against a physical table in

Finacle or a view in the Data Warehouse.

This is the single most consequential design decision in the architecture, and it directly answers the RFI's requirement for a prototype demonstrating integration *"via YES BANK Data Warehouse, or direct connectivity to source applications"*. Because controls bind to the canonical contract rather than to a physical schema, the same 78 controls run unchanged whether the Bank routes them through the Data Warehouse or connects them directly to core banking. Changing the route is a connector configuration change; it is not a re-write of the control library.

The eleven canonical entities are `customer`, `account`, `transaction`, `loan`, `journal_entry`, `employee`, `entitlement`, `vendor`, `vendor_payment`, `kyc_document` and `aml_alert`. Each declares a primary key, typed columns with required flags, and *entity dimensions* — branch, region, segment, product — carried from day one so that branch and regional analysis needs no retrofit.

The evidence pipeline

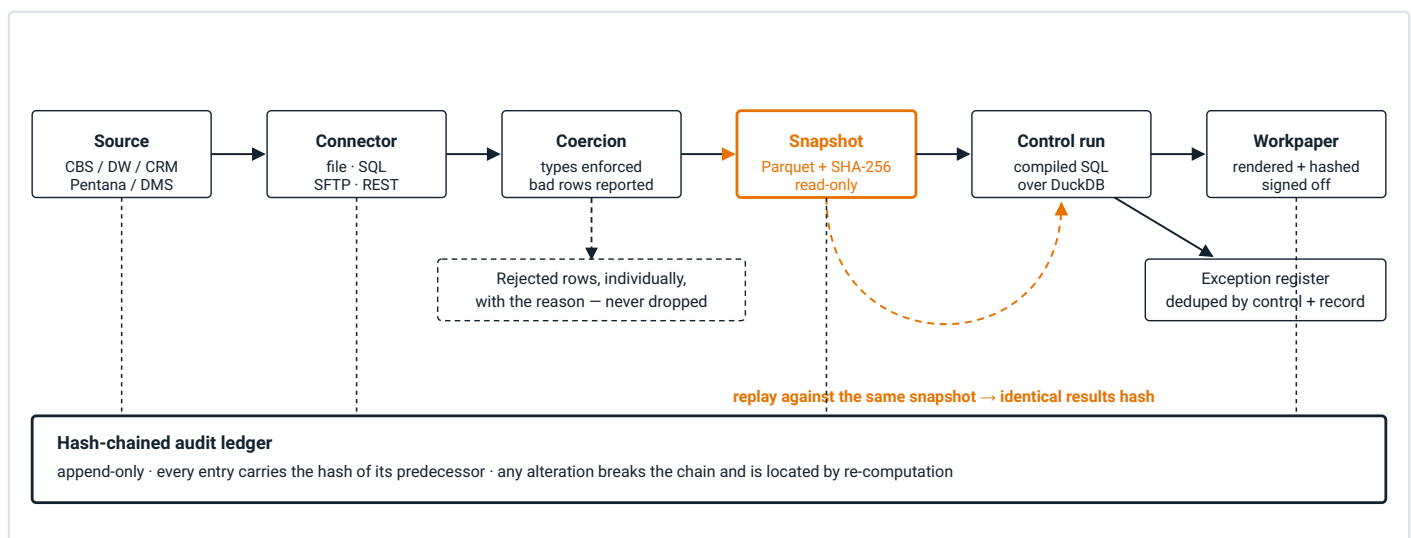


Figure 3 — The evidence pipeline. The orange path is the assurance argument: data becomes immutable and content-addressed *before* any control sees it, and a completed run can be replayed against the same snapshot to produce an identical results hash. Rows that fail type coercion are reported individually with their reason rather than silently discarded — a population that shrank without explanation is not auditable.

Retention, lineage and reconciliation

- **Lineage** AS-BUILT — every exception carries its run id; every run carries its snapshot id and the exact SQL executed; every snapshot carries per-table content hashes and row counts. An auditor can move from a number in a workpaper to the specific frozen dataset it came from without leaving the system.
- **Reconciliation reports** AS-BUILT — pull-level row counts, rejected-row detail and per-table content hashes support source-to-platform reconciliation. §3.2 asks for reconciliation reports;

the data to produce them is captured at ingestion.

- **Retention and archival** AS-PROPOSED — configurable retention with WORM archival of snapshots and ledger segments. Snapshots are already immutable and content-addressed, which is the hard half; the policy engine and archival scheduling are implementation-phase work.

7 Control architecture

Controls are the Bank’s intellectual property, not the vendor’s. The architecture reflects that: a control is a validated configuration object that the Bank can author, review, approve and version without a release, and without the vendor.

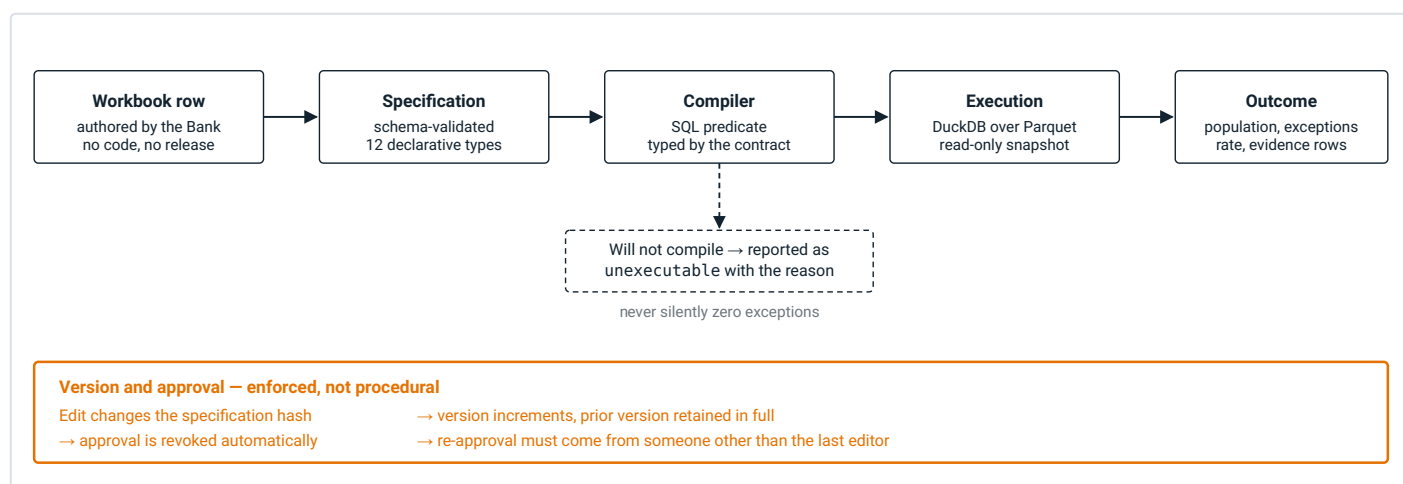


Figure 4 — A control from authoring to outcome. The dashed branch matters as much as the main path: a control the compiler cannot build is reported as unexecutable with its reason, and the run that contains it is marked inconclusive. A control library that silently reports zero exceptions is worse than no library at all.

The twelve control types

Type	Tests
Comparison	A column against a threshold, a value or another column, typed from the canonical contract rather than from the literal
Is_Null / Not_Null	Presence or absence of a value, treating a blank string and a null identically
Value_In_Set	Membership in a permitted list; a blank counts as outside the set and is flagged
Date_Order	Chronological relationship between two dates — a document verified before it was received

Type	Tests
Date_Diff	Elapsed days between two dates against a limit – TAT and ageing controls
Date_Age	Age of a record relative to the snapshot date – dormancy, staleness, expiry
Duplicate	Repeated key combinations, with blank keys excluded because three missing invoice numbers are a completeness issue, not a duplicate-payment issue
Group_Aggregate	Count, sum, min or max within a group against a limit – splitting, structuring, concentration
Group_Share	Concentration of a group as a share of the whole – vendor and channel dependence
Matrix_Lookup	Authority matrices: the level required for an amount against the level actually held. A missing amount takes the <i>highest</i> band, so an unrecorded sanction fails open
Expression	Free-form predicate, restricted to canonical columns and an allow-list of deterministic SQL functions; non-deterministic functions are refused so a control cannot return different rows on re-run

Reproducibility

AS-BUILT Re-executing a completed run against the same snapshot produces an identical results hash, and the verification replay deliberately does not write a second copy of the exceptions. The Bank can therefore prove to a regulator that a conclusion reached six months ago is reproducible today from the archived evidence, using the control version that was in force at the time.

8 AI and ML architecture

Two distinct capabilities are often conflated in RFI responses. This architecture keeps them separate because their governance requirements are different.

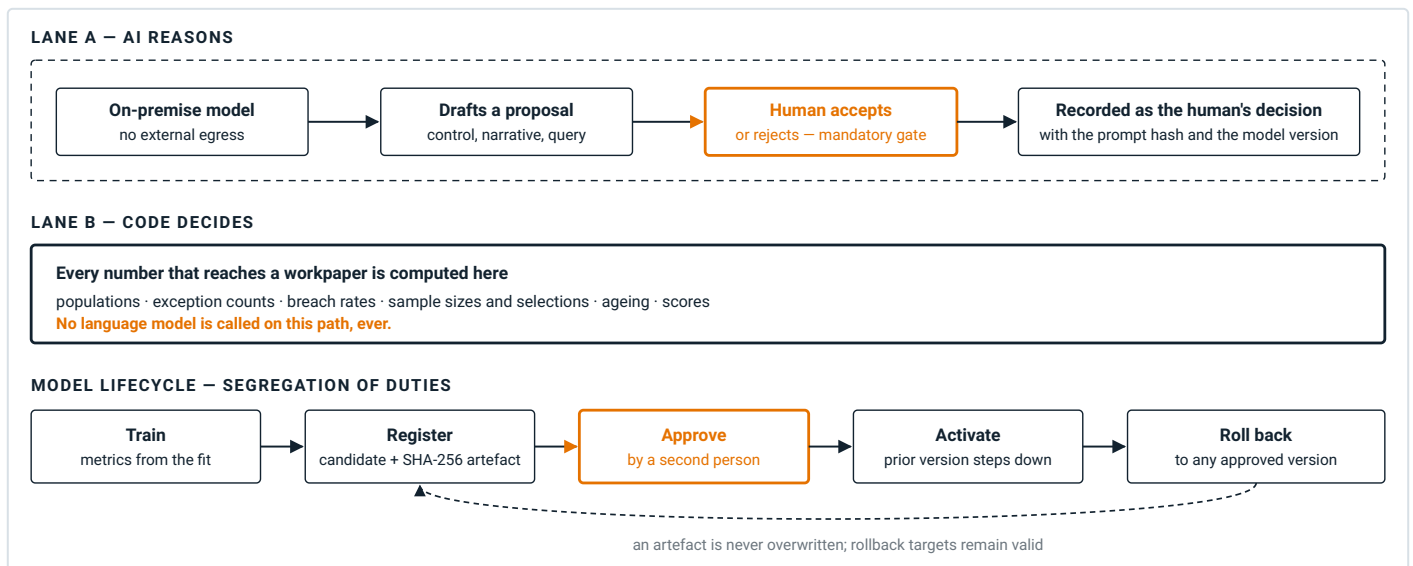


Figure 5 – AI governance. The orange gates are the two that a regulator will ask about: a model’s output cannot become a record without a named human accepting it, and a model cannot enter production on the say-so of the person who trained it. Lane B is drawn to make one negative claim explicit – the analytics path contains no model call.

Generative AI

AS-BUILT Inference runs on-premise. The provider layer parses every configured endpoint and refuses any URL that is not a loopback or private host, with a separate configuration guard that must be explicitly set to permit an external provider. A scheme-less host string that could be read as a remote endpoint is rejected rather than accepted by default. In the reference environment the model is served locally; in production the Bank chooses the model and the hosting node.

Generative capabilities in scope: drafting a control from a natural-language description (compiled, validated and previewed against real data before it can be saved), drafting findings and multi-level report narratives from recorded evidence, drafting auditee queries, and the policy assistant. The assistant answers strictly from an indexed corpus with per-document role-based access enforced at retrieval time, cites the document, version and page for every claim, surfaces contradictions where two documents disagree, and returns “I don’t know” rather than speculating when the corpus does not contain the answer.

Machine learning

AS-BUILT Two models, both governed by the registry shown above:

- **Exception triage** — a gradient-boosted classifier over 17 engineered features predicting the probability that an exception will be confirmed on review, blended with a documented heuristic so the score never becomes an unexplainable number. Trained on the audit function’s own

dispositions. In the reference environment it achieves a held-out AUC of 0.785, with precision and recall reported alongside; SHAP contributions accompany every score.

- **Anomaly detection** — an isolation forest over the same feature space, unsupervised so it needs no labelling history to be useful on day one.

Every score the platform acts on is written as an immutable decision record carrying the model key, version, inputs and reasoning — so the question “why did the model say that, on that day, with that model?” is answerable at review time.

AS-PROPOSED Drift, bias and accuracy monitoring with scheduled re-training proposals. The registry, versioning, approval and rollback machinery this depends on is already in place; the monitoring loop is implementation-phase work.

9 Security architecture

This section responds to §3.2 (Security & Compliance) and §5.5 (Security Features). The clause-by-clause response is in the Technical & NFR Response Matrix; what follows is the architecture behind it.

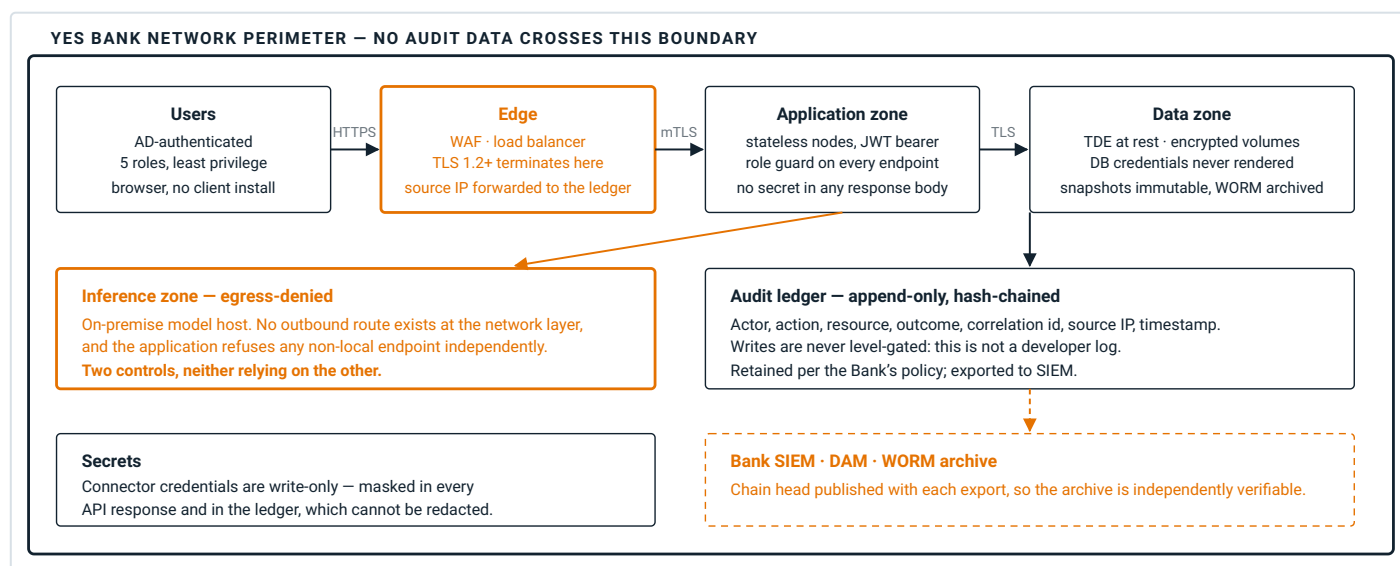


Figure 6 — Security zones and log egress. Two claims are drawn deliberately. First, the inference zone is isolated by *two independent* controls — a network with no outbound route and an application that refuses non-local endpoints — so neither is a single point of failure for data residency. Second, the ledger's chain head is published with each SIEM export, which is what makes the archived trail verifiable outside the platform.

Identity and access

Control	Status	Design
Role-based access control	AS-BUILT	Five roles – administrator, audit manager, auditor, reviewer, auditee. Every one of the 117 operations carries an explicit role dependency; there is no unguarded endpoint other than login and health.
Segregation of duties	AS-BUILT	Enforced in code at six gates: control approval, checklist approval, finding QA, action verification, extension approval, model promotion.
Password storage	AS-BUILT	PBKDF2-HMAC-SHA256 at 600,000 iterations with a per-user salt. Login performs equivalent work for a non-existent user, so response time does not disclose which usernames are valid.
Session management	AS-BUILT	Signed bearer token, 12-hour lifetime, <code>HttpOnly</code> and <code>SameSite=Strict</code> cookie with the Secure attribute configurable for TLS deployment.
Separate user-management module	AS-BUILT	Administration surface for provisioning, role assignment, activation and de-activation, with a guard preventing removal of the last active administrator.
AD / LDAP integration	AS-PROPOSED	§3.2 requires AD login. Delivered as a pluggable authentication provider – LDAP bind or SAML/OIDC against the Bank's identity provider, with group-to-role mapping. The authorisation model is unchanged; only the authentication step is replaced.
Multi-factor authentication	AS-PROPOSED	Delegated to the Bank's identity provider once AD/SSO is in place, which is the Bank's preferred pattern and avoids a second MFA estate.
User dormancy	AS-PROPOSED	Automatic de-activation after a configurable idle period. Last-login is already recorded per user.

Data protection

- **In transit** AS-PROPOSED – TLS 1.2+ terminating at the Bank's load balancer or WAF, with mutual TLS between application and data zones. §5.5 requires encryption in transit including between the application and interfacing systems; connector configurations support TLS-secured endpoints for SQL, SFTP and REST.
- **At rest** AS-PROPOSED – transparent data encryption at the database tier and encrypted volumes for the snapshot store. Handled at the platform tier rather than in application code, which is the pattern the Bank's existing estate already operates and audits.

- **Secrets** **AS-BUILT** — connector credentials are treated as write-only. They are masked in every API response and in the audit ledger, and a client that echoes a masked value back on edit does not overwrite the stored secret. This matters because the ledger is immutable: a credential written there could never be redacted.
- **PII** **PARTIAL** — the canonical model carries identity documents in masked form by design. Field-level masking policies by role, and DPDP-aligned subject-rights handling, are implementation-phase work.

Application security

The platform is developed against OWASP guidance, and the reference implementation has been through a full internal security review whose findings were remediated before this submission. Parameterised queries and an allow-list of canonical identifiers throughout; uploads are extension-restricted, size-capped and streamed, with the destination path confined to the upload directory; every list endpoint is bounded at both ends so pagination cannot be defeated; error responses do not leak internal state. Independent VAPT and source code review are scheduled as pre-go-live activities and their reports form part of the implementation deliverables (§5.5, §6).

10 Integration architecture

§3.2 requires APIs for integration with the Data Warehouse, core banking, CRM, Pentana and other source applications. The platform integrates in one of two directions, and the Bank can mix them per source.

Connector	Status	Use and configuration
SQL	AS-BUILT	Direct read against the Data Warehouse or a core-banking replica over a SQLAlchemy-supported driver — Oracle, PostgreSQL, SQL Server, MySQL. Per-table query or view mapping. This is the recommended primary route for the Bank's DW.
SFTP	AS-BUILT	Scheduled collection of extract files from a landing zone, key- or password-authenticated. The pattern most core-banking teams prefer because it requires no inbound access to the source.
File	AS-BUILT	A mounted drop directory, reads confined to that directory so a configuration value cannot traverse the host filesystem. Used for nightly extracts and for one-off audit data requests.

Connector	Status	Use and configuration
REST	AS-BUILT	For source applications exposing an API — CRM, Pentana, V-CIP platforms. Bearer or basic authentication, offset or cursor pagination, with paging that stops only on a genuinely empty page so a server-side page-size cap cannot silently truncate the population.

Mapping, drift and failure

Each source declares a mapping from its physical columns to canonical columns. At ingestion, values are coerced to the declared canonical types; a value that cannot be coerced rejects *that row* with a recorded reason and column, and the pull continues. Absent optional values are recorded as null rather than treated as failures. Schema drift — a column that disappears or changes type between pulls — is detected and reported against the pull rather than discovered later as a control that quietly stopped finding anything.

Outbound and platform APIs

AS-BUILT All 117 operations are documented in an OpenAPI specification served by the platform, covering ingestion, controls, runs, exceptions, sampling, workpapers, the audit lifecycle, workflow, AI services and administration. This allows the Bank to drive the platform from an orchestration tool, feed dashboards into an enterprise BI layer, or push findings into an existing GRC system. The Integration & API Documentation deliverable is generated from this specification, so it cannot drift from the implementation.

11 Deployment architecture

This is the section where the two layers differ most, so the difference is drawn explicitly rather than described.

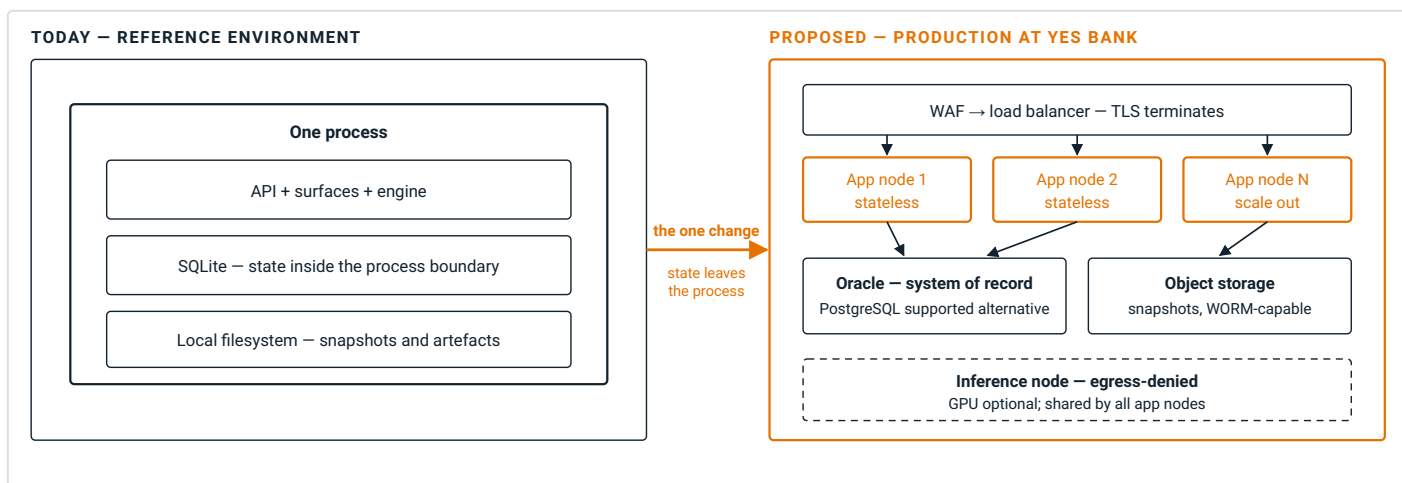


Figure 7 – What actually changes between the reference environment and production. The application tier is already stateless and token-authenticated, so scaling out is a deployment change rather than a re-write. The single architectural move is relocating state – the relational store to Oracle and the snapshot store to object storage. DuckDB is retained deliberately: it is the embedded analytical engine that reads snapshots, not the system of record.

Data centre and disaster recovery

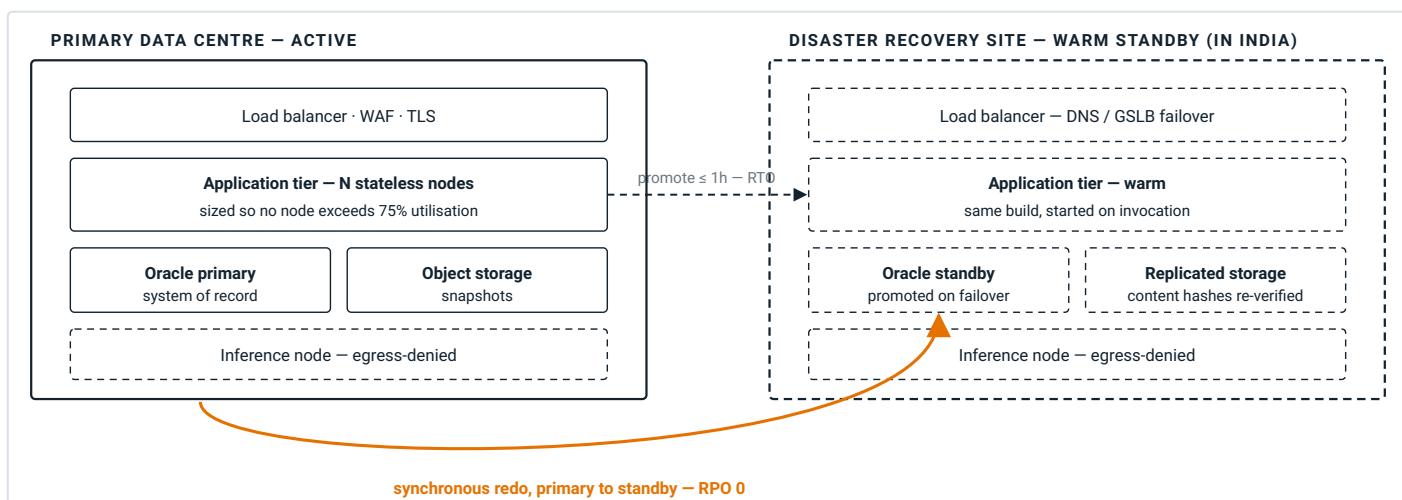


Figure 8 – DC and DR topology. A zero RPO is only achievable with synchronous replication, so that is what is proposed – the commitment is drawn on the diagram rather than asserted in prose. The one-hour RTO is met by a warm standby tier running the same build, with promotion and DNS failover as the recovery action. On failover the snapshot store's content hashes are re-verified, so the Bank knows its evidence survived the event intact.

Scaling and sizing

Target (§3.2)	Architectural response
5,000+ concurrent users	Stateless application tier scaled horizontally behind the load balancer; sessions are bearer tokens, so no sticky sessions and no shared session store. Read-heavy dashboard traffic served from the relational tier with a read replica.

Target (§3.2)	Architectural response
Response time < 2 seconds	Interactive requests are served from the relational store. Analytical work – control execution over millions of rows – runs against columnar Parquet through a vectorised engine and is asynchronous by design: a run is submitted and tracked, not waited on.
High-volume loads, large branch networks	Ingestion is per-source and parallelisable; snapshots are immutable so readers never contend with writers. Controls execute against columnar storage with predicate push-down.
Multi-entity, multi-process parallel execution	Runs are independent units keyed by pack and snapshot; several engagements execute concurrently without interaction, which is a consequence of pinning every run to frozen evidence.
Horizontal scaling for peak audit cycles	Application nodes added and removed without state migration. Peak audit periods are handled by scaling the tier rather than by capacity held idle year-round.
75% utilisation ceiling (§5.3)	Sizing in the BOQ is calculated to hold CPU, memory, storage and network below 75% at both DC and DR under the modelled peak.

Cloud portability

§5.3 requires that the solution be capable of being shifted from on-premises to the Bank's cloud provider. The architecture holds no on-premises assumption: the application tier is a stateless process, the relational store is accessed through a database abstraction supporting Oracle, PostgreSQL, SQL Server and MySQL, and the snapshot store is object storage. The same build deploys to on-premises, private cloud or public cloud; inference remains inside whichever boundary the Bank designates, and the residency guard travels with the application.

Environments

§3.2 requires separate UAT and DR environments and separate application and database servers. Four environments are proposed – Development, UAT, Production and DR – with separate application and database hosts in each of Production and DR. Because a snapshot is content-addressed, a dataset promoted from UAT to Production is provably the same dataset, which makes UAT sign-off meaningful rather than indicative.

12 Non-functional requirements

Summary position against §3.2. The clause-by-clause response, including all nineteen §5.5 security items and the 30-point §6 checklist, is in the Technical & NFR Response Matrix.

Requirement	As-built	As-proposed	Basis
Audit logs for every system interaction	MET	MET	Hash-chained append-only ledger over every state change, independently verifiable
Separate user-management module	MET	MET	Administration surface with role assignment and lifecycle controls
Data residency	MET	MET	No external egress path; inference locality enforced in code and at the network
AD integration for login	NOT MET	MET	Pluggable authentication provider; LDAP or SAML/OIDC with group-to-role mapping
Encryption in transit and at rest	NOT MET	MET	TLS at the edge and between zones; TDE and encrypted volumes at the data tier
Availability 99.9%	NOT MET	MET	Redundant application tier, clustered database, DR site – subject to the Bank's infrastructure SLAs
Response time < 2 s	PARTIAL	MET	Interactive paths are sub-second in the reference environment; formal certification on Bank infrastructure at UAT
5,000+ concurrent users	NOT MET	MET	Stateless horizontal scaling; to be evidenced by the performance testing certification
DR RPO 0 / RTO 1 hour	NOT MET	MET	Synchronous replication; warm standby with promotion and DNS failover
Horizontal scaling, parallel processing	PARTIAL	MET	Runs are already independent units; the tier becomes multi-node in production
APIs for DW, CBS, CRM, Pentana	MET	MET	Four connector types plus 117 documented REST operations
Real-time monitoring and API dashboards	NOT MET	MET	Metrics endpoint feeding the Bank's existing observability stack, plus in-

Requirement	As-built	As-proposed	Basis
			product operational dashboards
System log aggregation, SIEM	PARTIAL	MET	Ledger and application logs exist and are structured; the SIEM forwarder is implementation-phase
Reconciliation and MIS reporting	MET	MET	Pull reconciliation data captured at ingestion; exports in XLSX, CSV and PDF
Schema change support	MET	MET	Drift detection at ingestion; controls bind to the canonical contract, insulating them from physical change
Rules and workflow maintained without release	MET	MET	Controls and checklists are versioned configuration, approved in-product
Mobile sync < 60 s	NOT MET	ROADMAP	The UI is responsive; a native mobile client with offline sync is a roadmap item – see §14

13 Technology stack and licensing

Every third-party component is permissive open source – MIT, BSD or Apache-2.0. No copyleft licence is used anywhere in the stack, and no component carries a per-seat or per-core runtime licence. This directly supports the MeitY *Open-Source Software Policy, 2014* obligation in §5.2, and it means the Bank’s software cost for the platform itself is the Oracle licence it already provides under §5.3.

Layer	Component	Version	Licence	Role
Runtime	Python	3.13	PSF	Application runtime
API	FastAPI	0.115	MIT	HTTP layer, OpenAPI generation, dependency-injected role guards
API	Uvicorn	0.30	BSD	ASGI server

Layer	Component	Version	Licence	Role
API	Pydantic	2.9	MIT	Request and response validation
Persistence	SQLAlchemy	2.0	MIT	ORM and database abstraction — the layer that makes Oracle, PostgreSQL, SQL Server and MySQL interchangeable
Persistence	Oracle Database PROPOSED	19c+	Bank-provided	System of record in production (§5.3)
Persistence	PostgreSQL ALTERNATIVE	15+	PostgreSQL	Supported alternative where the Bank prefers open source
Analytics	DuckDB	1.5	MIT	Embedded vectorised engine executing controls over snapshots
Analytics	Apache Arrow / Parquet	21.0	Apache-2.0	Columnar evidence storage
Analytics	pandas / NumPy	2.2 / 2.2	BSD	Ingestion-time coercion and feature engineering
Machine learning	XGBoost	3.2	Apache-2.0	Exception triage classifier
Machine learning	scikit-learn	1.7	BSD	Isolation forest, scaling, evaluation
Machine learning	SHAP	0.51	MIT	Per-prediction explanations
AI	On-premise LLM runtime	—	Open source	Local inference; model selected with the Bank
Scheduling	APScheduler	3.11	MIT	Cron-driven pulls and runs, with expression validation
Reporting	ReportLab / openpyxl	4.2 / 3.1	BSD / MIT	PDF and Excel generation

Layer	Component	Version	Licence	Role
Security	PyJWT / cryptography	2.10 / 45.0	MIT / Apache-2.0	Token signing, hashing primitives
Front end	Vanilla ES modules	—	—	No framework and no build step — nothing to patch for framework CVEs, and the deployed asset is the reviewed source
Quality	pytest	8.4	MIT	649 automated tests plus 126 end-to-end HTTP checks

WHY NO FRONT-END FRAMEWORK

This is a deliberate architectural choice with a security consequence the Bank's application-security team will recognise: there is no build pipeline, no transitive npm dependency tree, and no gap between the code that was reviewed and the code that is served. Front-end supply-chain risk is removed rather than managed.

14 Product roadmap

§6 pairs the roadmap with the architecture in a single evaluation point, so the two are presented together. Phases are sequenced by dependency, not by preference: hardening precedes scale, and scale precedes the capabilities that assume it.

Phase	Theme	Delivered
Phase 0 Available now	Reference platform	Nine functional surfaces on a shared core; 78 controls across three packs; canonical model and four connectors; immutable evidence with reproducible runs; exception register with triage; statistical sampling; workpapers; audit lifecycle with enforced maker-checker; policy assistant; ML triage and anomaly detection with a governed model registry. Demonstrable in the prototype the RFI requests.

Phase	Theme	Delivered
Phase 1 Enterprise readiness	Security, identity and operations	AD/SSO with group-to-role mapping and MFA through the Bank's IdP; TLS and encryption at rest; source IP, hostname and interface capture in the ledger; SIEM forwarding with chain-head publication; retention and WORM archival policy; user dormancy; independent VAPT and source-code review with remediation.
Phase 2 Scale and resilience	Production topology	Migration to Oracle with object-storage snapshots; multi-node application tier behind the Bank's load balancer; DC/DR with synchronous replication and warm standby; performance testing to 5,000+ concurrent users with certification; monitoring and API performance dashboards; incident and change management integration.
Phase 3 Functional depth	Completing the nine initiatives	OCR and document-screening for KYC with extracted-versus-system comparison; alerting and response orchestration for CCM; intelligent exception grouping and root-cause suggestion; automated checklist triggering; entity clustering and segmentation; predictive risk dashboards; the remaining agentic capabilities – autonomous multi-step execution, dynamic scope adjustment and follow-on test triggering, each behind the same human approval gate.
Phase 4 Continuous assurance	Model operations and mobility	Drift, bias and accuracy monitoring with controlled re-training proposals; native mobile client with offline capture and sub-60-second synchronisation; expanded control library co-developed with the Bank's audit teams.

STATED PLAINLY

The functional matrix records 170 of 228 requirements as delivered today. The remaining 58 sit in Phases 1 to 4 above, and the largest concentration is in the Agentic AI initiative, where roughly half the requirements are implemented. We would rather the Bank evaluate that figure knowing exactly what it counts than discover it during implementation.

15 Assumptions and prerequisites

Prerequisites from YES BANK (§6, point 9)

- Hardware, operating system and Oracle Database licences per §5.3, sized to the BOQ submitted with the commercial response.
- Network connectivity from the application zone to each in-scope source system, and a landing zone where SFTP or file collection is the agreed route.
- Data dictionaries and sample extracts for each source, to complete the physical-to-canonical mapping.
- Identity provider integration details for AD/SSO, and the group-to-role mapping the Bank wishes to apply.
- Load balancer, WAF, SIEM and DAM integration endpoints and the applicable configuration standards.
- The Bank's IT and IS security policies, encryption standards and password policy, referenced in §5.1 as shared with the selected bidder.
- Nominated business owners per initiative for control-library sign-off, and UAT participants.
- DC and DR facilities with the network capacity to support synchronous database replication.

Assumptions

- Source systems are read-only from the platform's perspective; no write-back is in scope.
- The Data Warehouse is the primary integration route where a required entity is available there, with direct source connectivity used by exception.
- Control logic is the Bank's to own and amend; Protiviti provides the initial library and the authoring capability.
- Language model selection and hosting are agreed jointly, and inference remains within the Bank's boundary in all cases.
- Availability figures are for the application; the end-to-end figure depends on the Bank's infrastructure and network SLAs.

Exclusions

- Provision of hardware, operating system and Oracle licences (§5.3 places these with the Bank).

- Remediation of data quality within source systems; the platform reports it rather than corrects it.
- Changes to source applications to expose data not currently available.
- Third-party licence costs for systems the platform integrates with.

WHAT WE WOULD SHOW AT DEMONSTRATION

An end-to-end run in the reference environment: connect a source and pull it, watch the data become a content-addressed snapshot, execute a control pack against it, triage the exceptions, select a statistical sample, raise a finding through QA into a management action, generate the workpaper, then verify the ledger chain and re-run the whole thing to prove the same evidence produces the same answer. Approximately 40 minutes, with nothing pre-recorded.